

Lifting Manifolds to Mitigate Pseudo-Alignment in LLM4TS

Liangwei Nathan Zheng
Australian Institute for Machine
Learning
Adelaide University
Adelaide, Australia
liangwei.zheng@adelaide.edu.au

Wenhao Liang
Australian Institute for Machine
Learning
Adelaide University
Adelaide, Australia
wenhao.liang@adelaide.edu.au

Wei Emma Zhang
Australian Institute for Machine
Learning
Adelaide University
Adelaide, Australia
wei.e.zhang@adelaide.edu.au

Miao Xu
The University of Queensland
Brisbane, Australia
miao.xu@uq.edu.au

Olaf Maennel
School of Computer Science and
Information Technology
Adelaide University
Adelaide, Australia
olaf.maennel@adelaide.edu.au

Weitong Chen*
Australian Institute for Machine
Learning
Adelaide University
Adelaide, Australia
weitong.chen@adelaide.edu.au

Abstract

Pseudo-Alignment is a pervasive challenge in many large language models for time series (LLM4TS) models, often causing them to underperform compared to linear models or randomly initialised backbones. However, there is limited discussion in the community for the reasons that pseudo-alignment occurs. In this work, we conduct a thorough investigation into the root causes of pseudo-alignment in LLM4TS and build a connection of pseudo-alignment to the cone effect in LLM. We demonstrate that pseudo-alignment arises from the interplay of cone effect within pretrained LLM components and the intrinsically low-dimensional manifold of time-series data. In addition, we also introduce *TimeSUP*, a novel technique designed to mitigate this issue and improve forecast performance in existing LLM4TS approaches. TimeSUP addresses this by increasing the time series manifold to more closely match the intrinsic dimension of language embeddings, allowing the model to distinguish temporal signals clearly while still capturing shared structures across modalities. As a result, representations for time and language tokens remain distinct yet exhibit high cosine similarity, signifying that the model preserves each modality's unique features while learning their commonalities in a unified embedding space. Empirically, TimeSUP consistently outperforms state-of-the-art LLM4TS methods and other lightweight baselines on long-term forecasting performance. Furthermore, it can be seamlessly integrated into four existing LLM4TS pipelines and delivers significant improvements in forecasting performance. We have released our code in the following repository: https://github.com/lcurasLW/LLM4TS_Mitigate_Pseudo_Alignment.git

CCS Concepts

• Computing methodologies → Temporal reasoning; • Applied computing → Forecasting.

*Corresponding Author



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792280>

ACM Reference Format:

Liangwei Nathan Zheng, Wenhao Liang, Wei Emma Zhang, Miao Xu, Olaf Maennel, and Weitong Chen. 2026. Lifting Manifolds to Mitigate Pseudo-Alignment in LLM4TS. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792280>

1 Introduction

Large Language Model has been shown to have significant impacts on World-Wide-Web application such as real-time online decision making by Time Series analysis, ranging from online medical diagnosis [35, 36] to weather [18], traffic flow [33], and energy-load forecasting [21]. Recently, large language models (e.g., GPT-2 and LLaMa) have been repurposed for forecasting via prompting, LoRA fine-tuning, and LayerNorm adaptation, giving rise to various LLM4TS (Large Language Models for Time Series) techniques. However, because these models are exclusively pre-trained in text, their suitability for time series tasks has come under criticism [23, 34] and they argue that a replacement of LLM to linear backbone or random initialized weight LLM can win a lot and even draw to LLM backbone. This challenges the premise that pretrained language knowledge directly benefits time series forecasting. A recent work [34] attributed this ineffectiveness of LLMs to an empirical phenomenon termed “Pseudo-Alignment”. This refers to a situation where *model representations of time-series and language appear superficially aligned at the level of first-order statistics (e.g., mean, see Figure 2a, most right), yet their full distributions remain distinct, revealing a failure of true semantic alignment and a distortion of modality-specific features*. The persistence of pseudo-alignment has been shown to consistently degrade model performance, sometimes performing even worse than linear or randomly initialized backbones [23, 34]. Consequently, there has been limited systematic investigation into the mechanistic origins of pseudo-alignment in LLM4TS models. Closing this gap is essential not only for fair evaluation of current approaches but also for guiding future designs. It motivates two central questions that remain largely unanswered: (1) **What may be the root causes for pseudo-alignment?** and (2) **Which architectural modifications or training strategies can effectively activate the rich knowledge of LLMs for time**

series forecasting? Addressing these questions can provide a principled foundation for future research and help distinguish when, how, and why language modality can be genuinely useful in time series domains.

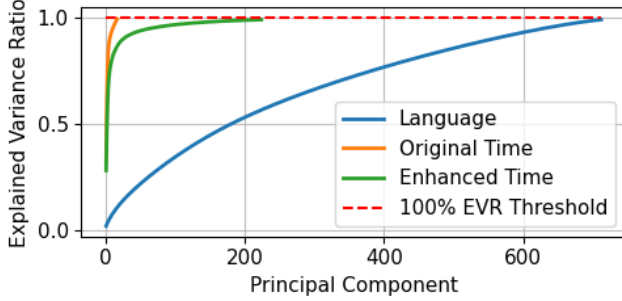


Figure 1: PCA Explained Variance Ratio plot for Language, Original, and Enhanced Time series

Although [34] empirically suggests that the performance of time series task come from the internal structure of time series modality instead of language due to the existence of pseudo-alignment, our systematic investigation further reveals the underlying causes of pseudo-alignment additional to the internal structure views and implement comprehensive exploratory analysis to find the following observations: (1) pseudo-alignment is a synthetic effect of the LLM cone effect [7] coupled with the inherently low-dimensional manifold of time-series data, (2) the architectural components primarily responsible triggering Pseudo-Alignment are the pretrained LayerNorm and Multi-Head Attention layers within the LLM, and (3) because of the cone effect, LLMs tend to “take easy route” by enforcing “perfect” alignment rather than recognizing the true low-dimensional structure of time-series. With these observations, we respect the rich language knowledge of LLM can positively contribute to time series analysis [11, 34] and theoretically show that low dimensionality of time series can cause pseudo-alignment. High dimensionality manifold of time series representation can prevent LLM from pseudo-alignment during the propagation of cone effect. Ideally, we expect the input time series to match the manifold dimension as language so as to activate the rich knowledge of LLM, and an ideal alignment exhibit preservation of distinct modality discrepancy while maintaining high cosine similarity in their shared subspace, thereby retaining modality-specific features and aligning shared information. The high discrepancy indicates the model can clearly tell the difference of modality distribution that provide unique modality feature and structure, which is highly correlated to modality gap phenomenon as discussed in [12]. The high cosine similarity at token-level indicates that the time series and language modality lives in a consistent and highly aligned embedding space [7].

In addition, we also introduce a simple, yet can broadly adapted module for many LLM4TS approaches to effectively address pseudo-alignment and boost the performance in long-term time series forecasting task. Based on observations (3), we proposed TimeSUP, a Time Manifold Enhancer, that elevates the manifold dimensionality of time-series data (originally 21 dimensions) toward that

of GPT-2’s language token manifold (712 dimensions), yielding strong and consistent performance improvements in our experiment. To demonstrate the effects of this dimensional enhancement, we conducted a probe experiment with PCA (see Figure 1). Results show that the enhanced representation raises the intrinsic manifold dimension of time series from 21 to 224 in order to preserve a 0.99 explained variance ratio, thereby significantly increasing the data manifold. This dimensional expansion allows the time series to occupy a higher-dimensional manifold that is more compatible with the pretrained LLM’s representational geometry. Further visualization in Figure 2 (second column) reveals that the pseudo-alignment issue is notably mitigated, while token similarity remains at relatively high values. This indicates that TimeSUP not only preserves local relational structures but also avoids the distortion of modality-specific features, offering extensible insights into modality alignment. Our findings suggest that dimension enhancement is a critical mechanism to unlock the latent capabilities of LLMs in non-linguistic domains. By mitigating pseudo-alignment, the proposed Time Manifold Enhancer provides a generalizable solution that can be integrated into a wide range of LLM4TS pipelines. This line of exploration opens new opportunities for bridging modality gaps and further harnessing the power of LLMs for time series understanding and beyond. Our main contributions can be summarized as follows:

- We are the first work to reveal pseudo-alignment problem from the perspective of data manifold dimensionality, revealing new insights of LLM4TS model. We also implement comprehensive experiment to show the impact of low dimensionality of time series.
- We proposed **TimeSUP**, an simple, yet effective way of reprogramming large language model for time series, to effectively address Pseudo-Alignment problem by raising the exact dimensionality of time series data. Similar learning paradigm can be extendable to other form of reprogramming large language model for time series.
- TimeSUP consistently outperforms the state-of-art LLM4TS baseline on various long-term forecasting dataset. Moreover, it is easily adaptable to other LLM4TS approach, achieving instant performance improvement over original baseline implementation.

2 Preliminary

2.1 Time Series Lies in a Lower-Dimensional Data Manifold

According to the manifold hypothesis [2], real-world data often reside on low-dimensional manifolds embedded within high-dimensional spaces. This suggests that complex observations can be described using only a few intrinsic dimensions. To examine this, we apply PCA to both time-series tokens $T_i \in \mathbb{R}^d$ and GPT-2 language tokens $L_i \in \mathbb{R}^d$, where $d = 768$. Our results show that time-series tokens (generated with a patch size of 16 and stride of 8) can be well-represented using just 21 principal components, whereas language tokens retain 712 out of 768 components. This indicates that time-series modality lie on a much lower-dimensional manifold than language modality [14].

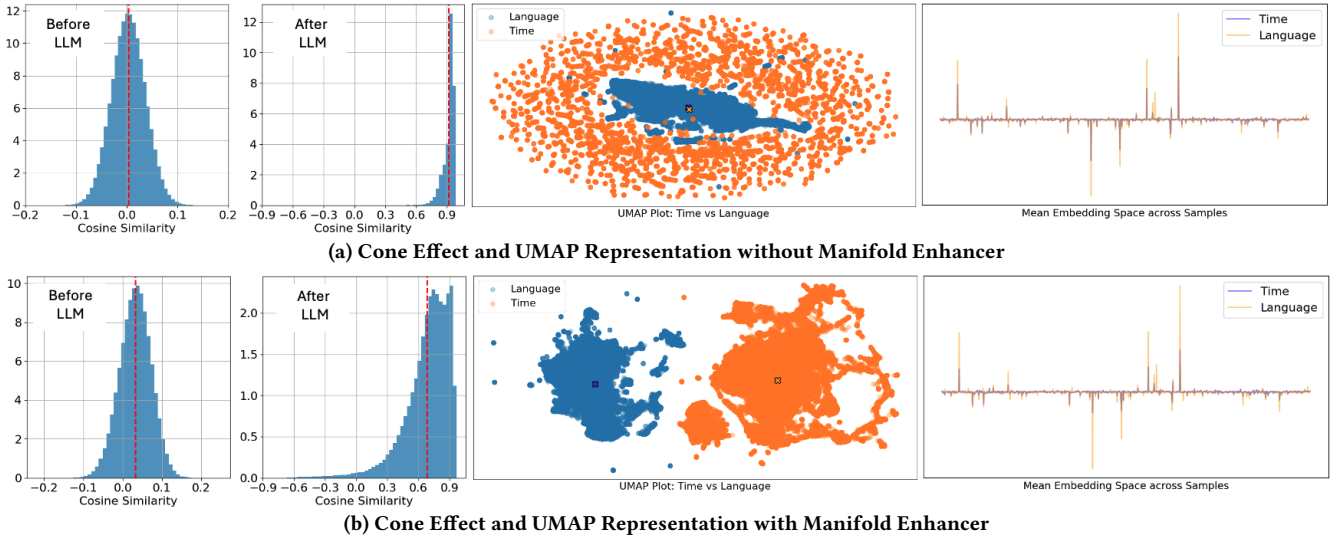


Figure 2: Comparison between the representation and similarity with and without TimeSUP: (1) Cosine similarity is determined by paring time series and language tokens. (2) Pseudo-Alignment is still preserved if time series token is fed in LLM. (3) Pseudo-Alignment phenomenon is mitigated by TimeSUP.

This observation helps explain why pretrained language models can still perform well on time-series tasks under the model reprogramming assumption: if the source model is trained on a more complex domain (language), it can be adapted to a simpler target domain (time series). In essence, successful cross-modal reprogramming relies on the pretrained model being expressive enough to capture the target distribution [5, 6, 31]. However, the simplicity of the time-series manifold may also limit the generalization ability of LLMs in time-series applications, as the representation of time series may be easier to manipulated than complex representation.

2.2 Cone Effect in Large Language Models

The Cone Effect refers to a phenomenon where token embeddings in LLMs collapse into a narrow cone, reducing their representational diversity [1, 7, 12]. Initially observed in language token embeddings, this effect causes tokens to become increasingly similar as cosine similarity values rise close to 1 after each LLM layer, whereas the tokens were initially well-dispersed in the feature space. This is largely due to architectural components like LayerNorm and Multihead Attention [22]. The cone effect can reduce the expressiveness of LLMs, as all tokens tend to appear positively correlated, even if they are semantically distinct. In the setting of multimodal learning, when inputs from different modalities are fed into the model, multimodal data may form separate cones with a gap in between due to data heterogeneity, which can positively impact downstream performance [12].

In the context of LLMs for time series (LLM4TS), we also clearly observe this cone effect. As shown in Figure 2a, the mean cosine similarity between language and time tokens is near zero and spans mostly in between -0.1 to 0.1, but significantly increases beyond 0.9 after LLM and most of token similarity are all positive. The UMAP visualization and mean representation plot also illustrate a

strong Pseudo-Alignment, where time-series representations collapse toward those of language in a centroid, rather than achieving meaningful cross-modal alignment. Ideally, we hypothesis that a well-aligned model should maintain distinct modality-specific features while aligning shared semantics as they are partially shared information across modality instead of perfectly mapping relationship, resulting in separate yet high token similarity in the shared embedding space [7, 12].

3 Understand Pseudo-Alignment

In this section, we investigate the underlying cause of Pseudo-Alignment in current LLM4TS methods. We consider GPT-2 with LayerNorm fine-tuning as the representative architecture for our empirical analysis, as it is the most commonly adopted backbone across many LLM4TS approaches [9, 9, 20, 38]. Our analysis focuses on the long-term time series forecasting task, utilizing the official implementation of OFA [38] as the baseline method for visualization, where time series tokens are directly fed into the LLM with only the LayerNorm layers fine-tuned. Through the analysis above, we understand that time-series data lives in a much lower-dimensional manifold space and exhibits a simpler structure than language. Thanks to the pretrained LayerNorm and multi-head attention layers, LLM can manipulate and redistribute the feature map of perceptual tokens (here, time-series) into the textual feature domain. However, the time-series manifold is too simple that the model “takes easy route”, and it learns to manipulate features toward perfect, 100% alignment rather than deeply distinguishing modality-specific and shared information. This manipulation can be considered as an implicit multimodal alignment from pretrained attention and layernorm weight [22]. Consequently, Pseudo-Alignment emerges as a synergistic effect between the cone effect in LLMs [7] and the inherently low-dimensional manifold of time series. This issue is especially detrimental to time series data, as its

simple structure makes it particularly vulnerable to such distortion manipulation.

To support our claim above, we show that the Pseudo-Alignment is caused by the low-dimensional data manifold under the condition of cone effect occurring naturally in LLM. Let

$$\epsilon_{ts} = [v_1, v_2, \dots, v_m], \quad v_i \sim \mathcal{N}(0, \sigma_{ts}^2) \quad (1)$$

$$\epsilon_l = [v_1, v_2, \dots, v_n], \quad v_j \sim \mathcal{N}(0, \sigma_l^2) \quad (2)$$

be the stochastic deviations induced by finite sampling from their respective modality distributions. Here m and n denote the intrinsic manifold dimensionalities of the time-series and language modalities. We assume that the time-series x_{ts} and language x_l are independently and identically distributed (i.i.d.) w.r.t their modality distributions and can be represented as the sum of the modality mean and sampling perturbations:

$$x_{ts} = \mu_{ts} + \epsilon_{ts}, \quad x_l = \mu_l + \epsilon_l \quad (3)$$

We quantify the cone effect by cosine similarity between time series and language modality in Figure 2. Thus, we can obtain the following theorem:

THEOREM 1. *When the manifold dimensionality $m \rightarrow 0$ and $n \rightarrow 0$, The cosine similarity tends to converge to the similarity between the mean of time series and language distribution, leading to pseudo-alignment. Formally, we can represent this pseudo-alignment process as follow:*

$$E[\cos(x_{ts}, x_l)] = E\left[\frac{\langle x_{ts}, x_l \rangle}{\|x_{ts}\| \|x_l\|}\right] = \frac{\mu_{ts} \mu_l}{\sqrt{\|\mu_{ts}\|^2 + m\sigma_{ts}^2} \sqrt{\|\mu_l\|^2 + n\sigma_l^2}} \quad (4)$$

In practice, as the cone effect occurs naturally in LLM, which the token-level similarity raises significantly after each layer of LLM. As $m \ll n$ and $m\sigma_{ts}^2$ can be easily ignored, if the cosine similarity is significantly increased due to cone effect, then Eq. 4 is encouraged to converge to high similarity between u_{ts} and the entire distribution of language, leading to pseudo-alignment.

Proof of Theorem 1. The numerator can be decomposed to $E[\langle x_{ts}, x_l \rangle] = E[(\mu_{ts} + \epsilon_{ts})(\mu_l + \epsilon_l)] = E[\mu_{ts}\mu_l] + E[\mu_l\epsilon_{ts}] + E[\mu_{ts}\epsilon_l] + E[\epsilon_l\epsilon_{ts}] = \mu_{ts}\mu_l$ as ϵ is zero-mean for both modality as assumption of normalization. For denominator, we would like to expand $\|x_{ts}\|$ into its mean and dimensionality. Thus, we can decompose $E[\|x_{ts}\|^2]$ into its mean and variance (see derivation in Appendix):

$$E[\|x_{ts}\|^2] = \|\mu_{ts}\|^2 + m\sigma_{ts}^2 \quad (5)$$

Next, we would like to obtain $E[\|x_{ts}\|]$ from Eq.5. Although $E[\|x_{ts}\|] = E[\sqrt{\|x_{ts}\|^2}] \leq \sqrt{E[\|x_{ts}\|^2]}$ by Jensen inequality, we show that $E[\|x_{ts}\|]$ can still be approximated as $\sqrt{E[\|x_{ts}\|^2]} = \sqrt{\|\mu_{ts}\|^2 + m\sigma_{ts}^2}$ in high dimensional space. Since we know $\|x_{ts}\|^2 = \|\mu_{ts}\|^2 + 2\mu_{ts}\epsilon_{ts} + \|\epsilon_{ts}\|^2$, and the variance $Var(\|x_{ts}\|^2)$ can be rewritten as (see derivation in Appendix):

$$Var(\|x_{ts}\|^2) = 4\sigma_{ts}^2 \|\mu_{ts}\|^2 + 2m\sigma_{ts}^4 \quad (6)$$

With results of Eq.6, we are able to evaluate the relative fluctuation of variable $\|x_{ts}\|^2$ is correlated to dimensionality m :

$$\frac{std(\|x_{ts}\|^2)}{mean(\|x_{ts}\|^2)} = \frac{\sqrt{4\sigma_{ts}^2 \|\mu_{ts}\|^2 + 2m\sigma_{ts}^4}}{\|\mu_{ts}\|^2 + m\sigma_{ts}^2} \quad (7)$$

In Eq. 7, the relative fluctuation of $\|x_{ts}\|^2 \sim \frac{\sqrt{m\sigma_{ts}^2}}{m\sigma_{ts}^2} = \frac{1}{\sqrt{m}}$ and $\frac{1}{\sqrt{m}} \in 0$ when m is large (in high dimensional manifold / space). This indicates the fluctuation of $\|x_{ts}\|^2$ away from its mean can be ignored. Therefore, $\|x_{ts}\|^2$ will concentrate around its mean $\|\mu_{ts}\|^2 + m\sigma_{ts}^2$. This makes $\|x_{ts}\| = \sqrt{\|x_{ts}\|^2} \approx \sqrt{E(\|x_{ts}\|^2)} = \sqrt{\|\mu_{ts}\|^2 + m\sigma_{ts}^2}$. Similarly, we apply the same process to language modality x_l and obtain $\|x_l\| = \sqrt{\|x_l\|^2} \approx \sqrt{E(\|x_l\|^2)} = \sqrt{\|\mu_l\|^2 + n\sigma_l^2}$. Proof done.

Theorem 1 clearly indicates that low dimensionality of time series can cause the pseudo-alignment under the cone effect condition. To address this, one approach is to explicitly constrain the development of the cone effect within each LLM layer using a dedicated loss function [7]. However, this may inadvertently suppress the implicit multimodal alignment between time and language, potentially deactivating the knowledge transfer capabilities of the LLM. An alternative solution is to enhance the manifold dimensionality of time series data prior to LLM. This dimension enhancement should ideally raise the manifold dimension to similar level of language distribution, thereby enabling the model to better capture the internal structure of time series. This complex structure obstacle “easy route” but implicitly force LLM to distinguish the time series tokens, leading to mitigation of Pseudo-Alignment. Moreover, because the cone effect arises organically from the LLM’s normalization and attention mechanisms, the enhanced time-series still benefit from this implicit alignment. In practice, the cone effect will continue to promote large angular separations and maintain cross-modal coherence, thereby preserving both the model’s ability to align modalities and its capacity to capture fine-grained unique feature of both modality.

4 Address Pseudo-Alignment

To this end, we proposed TimeSUP to address Pseudo-Alignment problem by increasing data manifold dimension of time series data to help language model to understand time series tokens. As shown in Figure 3, our proposed method follows the analysis from the preliminary and aims to achieve: (1) Find most relevant material as enhanced fusion material and (2) manifold dimension enhancement to mitigate Pseudo-Alignment. We expect the well-alignment should preserve distinct feature and remain high similarity.

Patch Time Series Embedding. Each input series x_{ts} with sequence length L will be first normalized to zero mean and unit standard deviation by reversible normalization layer. Then, it will be patched into several patches with patch size P and stride step S [19], producing number of patch $N = \lceil \frac{P-L}{S} \rceil + 1$. The patched time series patches will be linearly spanned to the shared language embedding space $\mathbb{R}^d$ to produce time series token $T_i \in \mathbb{R}^d$, where d is the hidden size of LLM backbone.

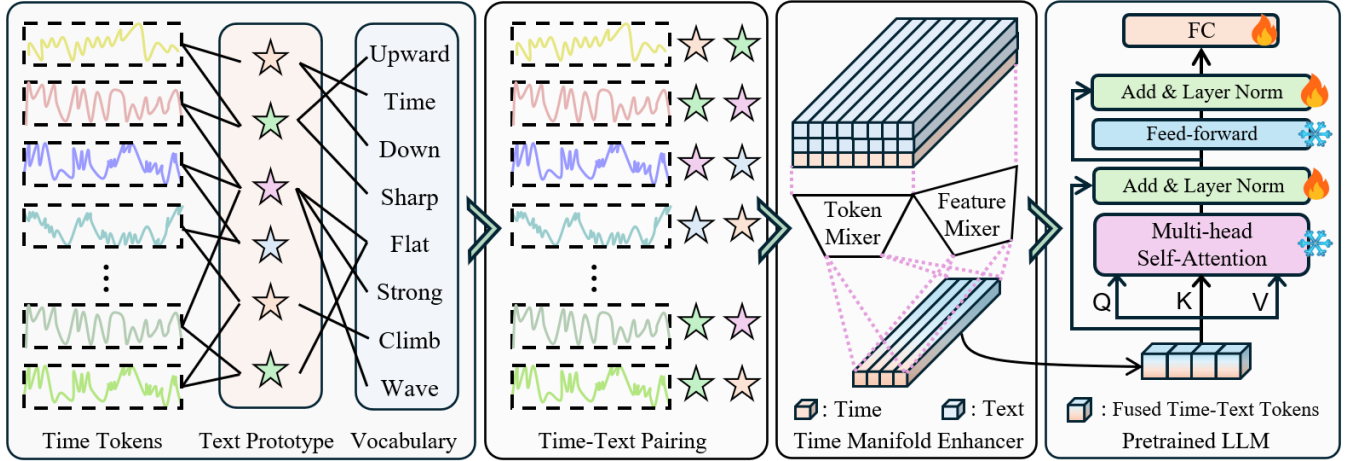


Figure 3: Overall Framework of Time Manifold Enhancer to address Pseudo-Alignment: (1) Text prototype is generated by a linear combination on the vocabulary and a one layer multihead attention is used to find text prototype best describing time series patches. (2) Each time patch will pick a top-k text prototype as the material to enhance dimension of time manifold. (3) Two MLP mixer are applied over the token dimension and feature dimension to compress and fuse the time-text tokens and generate the enhanced version of time tokens. (4) All fused time-text tokens will be sent to LLM for final prediction.

Top-K Text Prototype. Each time series patch contains strong temporal and localized pattern and we hypothesis that proper selection of language token can describe each time series patch (e.g. time is climbing). To help LLM understand the time series tokens from semantic level, we adapt text prototype [11] and find the most relevant language prototype to time series patches, where each time series token will be paired with Top-K language prototype to best describe the semantic feature about the time patch. Specifically, the text prototype $L_i^* \in \mathbb{R}^d$ is generated by linear combination of the entire vocabulary as illustrate in Figure 3. The number of prototype is set to 1000 by default.

Then, an asymmetric cross-attention is employed to find Top-K prototype that best describe for each time patch. We defined the query matrix $Q = TW_q$ from time series token $T \in \mathbb{R}^{N \times d}$ and key matrix $K = L^*W_k$ as prototype, where $L^* \in \mathbb{R}^{1000 \times d}$, $W_k, W_q \in \mathbb{R}^d$ and $Q, K \in \mathbb{R}^{d \times d}$. We calculate the attention weight by the dot-product of Q and K to obtain Top-K attention weight $A_k \in \mathbb{R}^{k \times N}$ as shown in Eq. 8. Note that we consider A_k to retrieve the TopK index to select the most relevant text prototype $L_K^* \in \mathbb{R}^{K \times N \times d}$ from L^* to each corresponding time token.

$$A_k = \text{TopK} \left(\text{Softmax} \left(\frac{QK^T}{\sqrt{d}} \right) \right) \quad (8)$$

Time Manifold Enhancer. Motivated by the previous analysis in which the time series lies in lower data manifold than language, The TopK prototypes are considered as a meaningful manifold enhancement material and a multimodal fusion with complex data and simple data will eventually produce a higher manifold dimension. Then, we leverage the Top-K language prototypes to enrich each time series token’s manifold dimension, bringing its manifold dimensionality closer to that of language tokens. We designed two lightweight MLPs, $M_c \in \mathbb{R}^{((K+1) \times N) \times n}$ operating across token dimension and $M_f \in \mathbb{R}^{d \times d}$ across feature channels to fuse each

time token with its selected prototype. Specifically, L_K^* will be concatenated with T to form a time-text pairs $T_t \in \mathbb{R}^{((K+1) \times N) \times d}$. We then compress T_t into a fixed sequence length n (e.g. 8 or 16) fused representation $T^* \in \mathbb{R}^{n \times d}$ by M_c . Next, the fused representation is then fed into M_f for further feature fusion as shown in Eq. 9.

$$T^* = M_f(M_c^T T_t)^T \quad (9)$$

This multimodal fusion substantially elevates the intrinsic dimensionality of the time series manifold: under a PCA probe, the number of time series principle component increases from 21 to 224 (compared to 712 for language tokens in GPT-2) as shown in Figure 1. We find this is particularly useful for not only manifold enhancement, but also improving the speed for LLM forward as the number of tokens are constrained to a certain number. By grafting the high-dimensional structure of language onto time-series data, our time enhancer produces richer token embeddings that preserve modality-specific distinctions while maintaining strong alignment on shared semantic features, ultimately boosting downstream performance and activating pretrained language knowledge and avoid pseudo-alignment problem.

5 Related Work

5.1 Lightweight Time Series Models

Most of time series forecasting approaches are designed specific tasks and trained from scratch. Typically, we can categorized the time series model into the following architecture: RNN-, CNN-, Transformer- and MLP-based methods. RNN-based models learned temporal feature by recursively temporal hidden states [3, 4, 27]. CNN-based model, on the other hand, learn a series of kernels to slide over the time series and extract temporal and localized feature [10]. However, RNN- and CNN-based models usually suffer from the long-term dependency as the architecture focus more on

Dataset	TimeSUP		FSCA		CALF		S2IP		TimeLLM		UniTime		OFA		TimeMixer		TimesNet		iTransformer		
Pred Length	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	
ETTh1	96	0.376	0.400	<u>0.381</u>	<u>0.404</u>	0.386	0.410	0.394	0.415	0.383	0.412	0.397	0.418	0.392	0.407	0.388	0.402	0.384	0.402	0.386	0.405
	192	0.408	0.427	<u>0.416</u>	<u>0.419</u>	0.431	0.430	0.443	0.445	0.424	0.434	0.434	0.439	0.432	0.430	0.441	0.429	0.436	0.429	0.441	0.436
	336	0.429	0.436	0.455	0.447	0.477	0.450	0.456	0.456	<u>0.437</u>	0.453	0.468	0.457	0.441	<u>0.438</u>	0.484	0.458	0.638	0.469	0.487	0.458
	720	0.433	0.456	<u>0.455</u>	0.471	0.473	<u>0.466</u>	0.471	0.478	0.458	0.484	0.469	0.477	0.468	0.471	0.498	0.482	0.521	0.500	0.503	0.491
	Avg	0.412	0.430	<u>0.426</u>	<u>0.435</u>	0.441	0.439	0.441	0.448	<u>0.426</u>	0.445	0.442	0.448	0.433	0.436	0.452	0.442	0.495	0.450	0.454	0.447
ETTh2	96	0.293	0.346	0.300	0.351	0.297	0.351	0.300	0.354	0.295	0.353	0.295	0.345	0.311	0.366	<u>0.294</u>	0.350	0.340	0.374	0.297	<u>0.349</u>
	192	0.342	0.382	<u>0.348</u>	<u>0.387</u>	0.369	0.388	0.370	0.397	0.410	0.421	0.374	0.394	0.391	0.412	0.372	0.392	0.402	0.414	0.380	0.400
	336	<u>0.369</u>	<u>0.411</u>	0.365	0.405	0.422	0.426	0.401	0.426	0.423	0.441	0.415	0.427	0.386	0.417	0.386	0.414	0.452	0.452	0.428	0.437
	720	0.407	0.438	0.407	<u>0.444</u>	0.421	0.450	0.428	0.455	0.455	0.467	0.425	0.444	0.433	0.456	<u>0.419</u>	0.444	0.462	0.468	0.427	0.445
	Avg	0.353	0.394	<u>0.355</u>	<u>0.397</u>	0.376	0.404	0.374	0.408	0.396	0.420	0.378	0.403	0.380	0.412	0.368	0.400	0.414	0.427	0.383	0.407
ETTm1	96	0.296	0.350	<u>0.301</u>	<u>0.351</u>	0.324	0.353	0.300	0.353	0.304	0.361	0.322	0.363	0.310	0.355	0.320	0.357	0.388	0.375	0.334	0.368
	192	0.332	0.369	0.332	<u>0.371</u>	0.369	0.376	<u>0.333</u>	0.377	0.358	0.384	0.366	0.387	0.351	0.390	0.361	0.381	0.374	0.387	0.377	0.391
	336	0.360	0.385	0.365	<u>0.391</u>	0.407	0.399	<u>0.362</u>	0.396	0.364	0.397	0.398	0.407	0.373	0.403	0.390	0.404	0.410	0.411	0.426	0.420
	720	<u>0.411</u>	0.418	0.421	<u>0.425</u>	0.472	0.436	0.407	0.429	0.424	0.431	0.454	0.440	0.423	0.432	0.454	0.441	0.478	0.450	0.491	0.459
	Avg	<u>0.350</u>	0.380	0.354	<u>0.385</u>	0.393	0.391	0.349	0.389	0.363	0.389	0.385	0.399	0.364	0.395	0.381	0.395	0.400	0.406	0.407	0.410
ETTm2	96	<u>0.174</u>	0.264	0.169	0.260	0.178	0.255	0.181	0.269	0.178	0.269	0.183	0.266	0.188	0.269	0.175	<u>0.258</u>	0.187	0.267	0.180	0.264
	192	0.233	<u>0.300</u>	<u>0.237</u>	0.306	0.243	0.298	0.257	0.323	0.241	0.313	0.251	0.310	0.251	0.309	0.251	0.309	0.251	0.312	0.249	0.309
	336	<u>0.280</u>	0.338	0.276	0.328	0.303	0.338	0.293	0.346	0.287	<u>0.337</u>	0.319	0.351	0.307	0.346	0.298	0.340	0.321	0.351	0.311	0.348
	720	0.358	0.381	0.366	0.390	0.403	0.396	0.367	0.396	<u>0.364</u>	<u>0.401</u>	0.420	0.410	0.426	0.417	0.391	0.396	0.408	0.403	0.412	0.407
	Avg	0.261	0.323	<u>0.262</u>	0.321	0.282	<u>0.322</u>	0.279	0.331	0.267	0.330	0.293	0.334	0.293	0.335	0.275	0.325	0.291	0.333	0.288	0.332
Illness	24	<u>1.942</u>	0.835	2.105	0.908	2.271	1.008	2.622	1.113	1.921	<u>0.902</u>	2.346	0.954	2.205	0.982	2.241	0.986	2.317	0.934	2.347	1.731
	36	<u>1.952</u>	0.948	2.066	0.962	2.365	1.029	2.515	1.098	1.992	0.918	1.998	<u>0.912</u>	2.166	0.969	1.863	0.903	1.972	0.920	2.468	0.998
	48	1.738	0.886	1.902	0.906	2.395	1.046	2.530	1.066	2.187	1.031	<u>1.979</u>	<u>0.912</u>	2.220	0.963	2.402	1.088	2.238	0.940	2.489	1.016
	60	1.907	<u>0.942</u>	2.341	1.016	2.415	1.056	2.329	1.065	2.125	0.992	2.109	0.938	2.246	1.010	2.156	0.983	<u>2.027</u>	0.928	2.471	1.065
	Avg	1.885	0.902	2.103	0.948	2.361	1.035	2.387	1.052	<u>2.056</u>	0.961	2.108	<u>0.929</u>	2.209	0.981	2.166	0.990	2.139	0.931	2.444	1.203
Weather	96	0.151	0.200	<u>0.152</u>	0.201	0.163	0.203	0.157	0.209	0.154	<u>0.199</u>	0.171	0.214	0.157	0.208	0.163	0.209	0.172	0.220	0.174	0.214
	192	0.197	0.243	0.197	<u>0.244</u>	0.210	0.248	0.200	0.251	0.214	0.254	0.217	0.254	<u>0.200</u>	0.246	0.208	0.250	0.219	0.261	0.221	0.254
	336	0.248	0.284	0.253	0.289	0.266	0.290	0.253	0.288	0.273	0.291	0.274	0.293	<u>0.250</u>	0.290	0.251	<u>0.287</u>	0.280	0.306	0.278	0.296
	720	0.330	<u>0.340</u>	<u>0.332</u>	<u>0.341</u>	0.347	0.344	0.338	0.333	0.364	0.350	0.351	0.343	0.330	0.342	0.339	0.341	0.365	0.359	0.358	0.349
	Avg	0.231	0.266	<u>0.233</u>	<u>0.269</u>	0.246	0.271	0.237	0.270	0.251	0.274	0.253	0.276	0.234	0.271	0.240	0.271	0.259	0.287	0.258	0.278
Traffic	96	0.369	0.266	<u>0.372</u>	0.255	0.433	0.292	0.379	0.274	0.388	<u>0.261</u>	0.491	0.313	0.393	0.288	0.462	0.285	0.593	0.321	0.395	0.268
	192	0.391	<u>0.277</u>	0.399	0.284	0.451	0.297	<u>0.397</u>	0.282	0.409	0.272	0.503	0.328	0.407	0.289	0.473	0.296	0.617	0.336	0.417	0.276
	336	0.408	<u>0.287</u>	0.413	0.273	0.467	0.302	<u>0.410</u>	0.289	0.438	0.299	0.527	0.347	0.446	0.296	0.498	0.296	0.629	0.336	0.433	0.293
	720	0.435	<u>0.298</u>	<u>0.440</u>	0.291	0.485	0.323	0.445	0.301	0.462	0.301	0.550	0.371	0.469	0.312	0.506	0.313	0.640	0.350	0.467	0.302
	Avg	0.400	<u>0.282</u>	<u>0.406</u>	0.276	0.459	0.303	0.407	0.287	0.424	0.283	0.518	0.340	0.428	0.296	0.484	0.297	0.620	0.336	0.428	0.284
ECL	96	0.128	0.221	0.133	0.228	0.167	0.254	<u>0.131</u>	<u>0.226</u>	0.172	0.265	0.196	0.287	0.137	0.236	0.153	0.247	0.168	0.272	0.148	0.240
	192	0.151	0.247	0.157	<u>0.250</u>	0.178	0.264	0.158	0.252	0.182	0.279	0.199	0.291	<u>0.153</u>	0.253	0.166	0.256	0.184	0.289	0.162	0.253
	336	0.170	0.269	0.176	0.281	0.196	0.282	0.183	0.280	0.195	0.288	0.214	0.305	0.183	0.280	0.185	<u>0.277</u>	0.198	0.330	0.178	0.269
	720	0.207	0.299	0.215	<u>0.302</u>	0.232	0.312	0.210	0.306	0.233	0.320	0.254	0.335	<u>0.209</u>	0.308	0.225	0.310	0.220	0.320	0.225	0.317
	Avg	0.164	0.256	<u>0.170</u>	<u>0.265</u>	0.193	0.278	<u>0.170</u>	0.266	0.195	0.288	0.216	0.304	<u>0.170</u>	0.269	0.182	0.272	0.192	0.295	0.178	0.270
Wins	60		13		2		2		2		0		0		2		1		0		

Table 1: Full Results Long-term Forecasting with LLM4TS: Bold Text indicate the best performer and Underline Text indicate the second best performer. We count the number of wins by the number of best performer.

local pattern. Therefore, recent works take advantages of global receptive field of Transformer for long-term feature extraction such as PatchTST [19], iTransformer [17], FedFormer [37], AutoFormer [29]. Furthermore, some researches argue that MLP-based method (e.g. DLinear [32], TimesNet [28], TimeMixer [26], FITS [30]) can even outperform transformer-based models and significantly simplified the parameter size. (Bold Text indicate the selected baselines in experiments)

5.2 Large Language Models for Time Series

Recently, large language models [24, 25] have been proved to have capability on strong generalization to time series analysis. Gruver et al. [8] convert time series as a series of digit token for zero-shot forecasting. OFA Zhou et al. [38] reprogrammed GPT-2 to adapt time series multi-tasks by finetuning the LayerNorm

layer. TimeLLM [11] employed prompts and cross attention to find the text tokens to best describe temporal feature from vocabulary. CALF [15] leverages LoRa finetuning and textual-temporal consistent loss to achieve model reprogramming. S2IP [20] decompose time series and align the language tokens to STL components. UniTime [16] attempt to train a unifying forecaster for time series by dataset description prompts and temporal feature of time. TimeCMA [13] trained an individual time series encoder alongside a pretrained LLM for language-time similarity retrieval. FSCA [9] introduce GNN to generate time-text interaction between time and text modality. (Bold Text indicate the selected baselines in experiments)

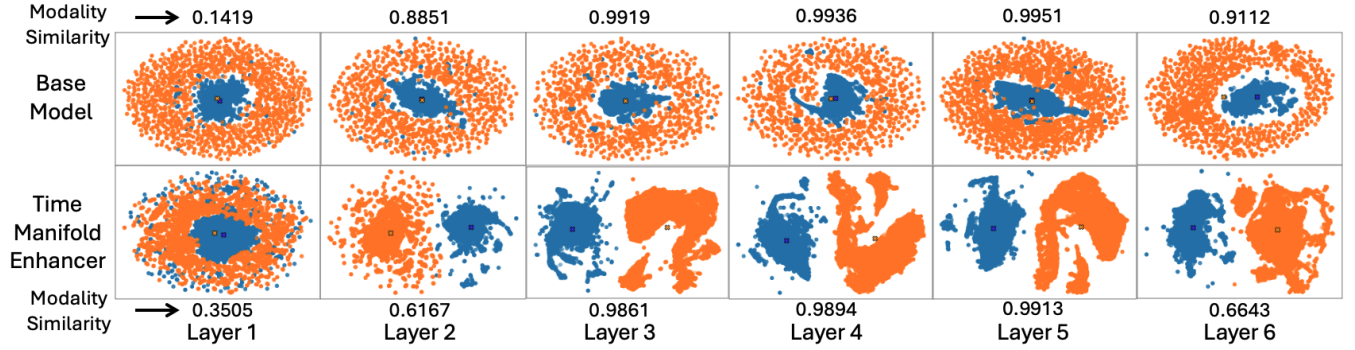


Figure 4: Representation in Layers: the cosine similarity is calculated between time and language tokens after each layer, **Blue** is language and **Orange** is time series. LLM starts to recognize time series token after layer 2 as two modalities are separable while maintain high cosine similarity.

6 Experiment

Set-up We evaluated TimeSUP on widely used benchmarking dataset for long-term forecasting, ETT(4 subsets), Illness, Weather, Traffic and Electricity [34, 38] for comprehensive comparison. All baseline are reproduced based on the best parameters from the original paper or github repository. The evaluation metrics include mean square error (MSE) and mean absolute error(MAE). Our ablation study is implemented on ETTh1 with 720 prediction length. Each Experiment were evaluate on $4 \times$ RTX 4090 24GB and $4 \times$ A100 40GB and default random seed from OFA [38]. All details parameter set-up for our proposed method is shown in Appendix. We will release our github repository upon the acceptance of paper.

In addition, we also visualize time and language tokens in layers of base model and base model with TimeSUP to show the impact of TimeSUP shown in Figure ?? . Moreover, we implement two additional ablation experiment to investigate the adaptability of Time Enhancer module in TimeSUP and what component in LLM causes Pseudo-Alignment as shown in Table 2.

6.1 Primary Results

As shown in Table 1, our proposed method outperforms other LLM4TS and typical lightweight model in 8 long-term forecasting dataset. In particular, our proposed method wins 60/80 on accross baseline models respectively. Moreover, those LLM4TS baseline can not consistently wins over lightweight model in Weather and ECL due to the harm of Pseudo-Alignment problem in those LLM4TS approach. TimeSUP, on the other hand, addresses Pseudo-Alignment effectively by increasing the dimension of time manifold, leading to superior improvement over LLM4TS and significant improvement over lightweight models. Although other LLM4TS approaches, such as S2IP and TimeLLM, are competitive to TimeSUP in ETTm1 and ETTm2, TimeSUP still win them consistently on ETTh1, ETTh2 and other large scale dataset. We show more detailed results with standard deviation in Appendix Table 5 that shows TimeSUP is stabler than other LLM4TS method.

6.2 Pseudo-Alignment in LLM Layers

To investigate the progression of pseudo-alignment across the LLM’s depth, we plot UMAP embeddings of language and time

series tokens alongside their layer-wise cosine similarity (Figure 4) by implementing visualization on ETTh1-720 and 6 layer GPT-2. The base model is GPT-2 fed only with time series tokens and finetuning layernorm layer. In the base model, cosine similarity surges to nearly 1 by the first layer and remains above 0.9 through all subsequent layers, indicating an almost complete collapse of the two modalities. Indeed, both the base and our proposed methods exhibit this centroid collapse at layer, where language and time series tokens become indistinguishable. However, from layer 2 onward, our approach diverges: the UMAP projections show that time series embeddings begin to fan out and map onto the language manifold without full collapse, and the cosine similarity, while still rising. This pattern demonstrates that our method successfully mitigates pseudo-alignment, preserving time series internal structure, while retaining enough implicit cone effect alignment to support cross-modal coherence. In addition, the increase in cosine similarity of TimeSUP proceeds more gradually than in the base model, and the two modalities remain clearly separable, indicating that the model actively distinguishes time-series tokens rather than collapsing them into the language centroid for a “perfect” alignment. As the network deepens, TimeSUP language and time representations gradually settle into their own distinct cones. Although their cosine similarity continues to rise, it plateaus at approximately 0.6643 by the final layer. This behavior is exactly what one would expect from a well-generalized multimodal model: it preserves each modality’s unique structure while maintaining substantial cross modal coherence, in line with the analysis in preliminary section. This explains that the high similarity in hidden layer from TimeSUP come from the actual alignment between tokens instead of the distribution centroid point.

6.3 Is the Enhancer Useful for other LLM4TS Model?

By integrating our TimeSUP enhancer into several leading LLM4TS methods—namely CALF, S2IP, UniTime and TimeLLM, we ensure that only minimal modifications are applied: the Time Manifold Enhancer is inserted immediately before the LLM backbone, and, where applicable, the existing text-prototype selection mechanism remains unchanged. OFA serves as the underlying backbone for

Model	CALF				S2IP				TimeLLM				UniTime				OFA				
Enhancer?	✓		✗		✓		✗		✓		✗		✓		✗		✓		✗		
Pred Length	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	
ETTh1	96	0.386	0.397	0.386	0.410	0.383	0.413	0.394	0.415	0.388	0.414	0.383	0.412	0.384	0.410	0.397	0.418	0.367	0.400	0.392	0.407
	192	0.427	0.422	0.431	0.430	0.408	0.428	0.433	0.445	0.412	0.433	0.424	0.434	0.430	0.435	0.434	0.439	0.408	0.427	0.432	0.430
	336	0.473	0.449	0.477	0.450	0.422	0.442	0.456	0.456	0.428	0.447	0.437	0.453	0.470	0.447	0.468	0.457	0.429	0.436	0.441	0.438
	720	0.468	0.465	0.473	0.466	0.457	0.474	0.471	0.478	0.453	0.478	0.458	0.484	0.447	0.461	0.469	0.477	0.433	0.456	0.468	0.471
	Avg	0.439	0.433	0.441	0.439	0.418	0.439	0.441	0.448	0.420	0.443	0.426	0.445	0.432	0.438	0.442	0.448	0.412	0.430	0.433	0.436
Illness	24	2.226	0.961	2.271	1.008	2.136	0.979	2.622	1.113	1.867	0.851	1.921	0.902	2.306	0.934	2.346	0.954	1.942	0.835	2.205	0.982
	36	1.831	0.848	2.365	1.029	2.183	0.990	2.515	1.098	1.883	0.879	1.992	0.918	2.003	0.922	1.998	0.912	1.952	0.948	2.166	0.969
	48	1.815	0.860	2.395	1.046	2.158	0.976	2.530	1.066	1.789	0.840	2.187	1.031	2.139	0.923	1.979	0.912	1.738	0.886	2.220	0.963
	60	1.892	0.879	2.415	1.056	2.085	1.009	2.329	1.065	1.852	0.875	2.125	0.992	2.073	0.934	2.109	0.938	1.907	0.942	2.246	1.010
	Avg	1.941	0.887	2.361	1.035	2.141	0.989	2.387	1.052	1.848	0.861	2.056	0.961	2.130	0.928	2.108	0.929	1.885	0.902	2.209	0.981

Table 2: Comparison Baselines with and without Manifold Enhancer: Bold indicate the improvement over original model

TimeSUP, so no additional alterations are needed. As reported in Table 2, this straightforward adaptation yields an average reduction in MSE of approximately 11% on the Illness dataset and 2% on ETTh1 across all tested configurations. The most pronounced gains appear in OFA and S2IP: on ETTh1, S2IP+TimeSUP achieves a 3% lower MSE and a 2% lower MAE compared to vanilla S2IP, while OFA+TimeSUP obtains a 4.8% reduction in MSE and a 1.3% reduction in MAE relative to OFA. This improvement is also consistent in Illness dataset. These improvements largely reflect the effectiveness of each model’s text-prototype selection when coupled with our enhancer. Conversely, TimeLLM shows the smallest performance increase on both ETTh1 and Illness; we attribute this to the relatively large number of prompt tokens in TimeLLM, many of which may be redundant for representing local temporal structure despite their utility for reasoning. Notably, the most competitive approach, FSCA [9], implicitly follows a similar idea to TimeSUP by enhancing the dimensionality of time series tokens due to the GNN information passing, although this is not explicitly discussed in their work. However, adapting TimeSUP within FSCA’s framework required substantial modifications to their implementation; therefore, we exclude FSCA from this comparison.

6.4 What Components in LLM Manipulate the Feature Map of Time Series?

This section empirically investigates how pretrained LayerNorm and Multi-Head Attention (MHA) contribute to Pseudo-Alignment of time series tokens. Building on our hypothesis that these components “take easy route” manipulate time-series embeddings to induce Pseudo-Alignment, we conduct an ablation study on a 6-layer GPT-2, as illustrated in Figure 5. We examine representation of time and language tokens under four conditions for combination setting: Pretrained–Frozen (PF), Pretrained–fineTuned (PT), Random–Frozen (RF), and Random–fineTuned (RT). This experiment will reveal how the pretrained language knowledge in multihead attention and layernorm layer impact Pseudo-Alignment and forecasting performance. Across all models with randomly initialized LN or MHA, we observe a pronounced separation between time series and language embeddings. By contrast, any model incorporating pretrained LN or MHA exhibits a strong tendency toward

pseudo-alignment, replicating the behavior shown in Figure 4. In particular, the combination of LN-PT and MHA-PT produces the most severe pseudo-alignment: these pretrained components reshape the time series feature manifold toward the language domain, while LN-PT dynamically modulates this shift to facilitate manipulation by the MHA-PF layers, as in OFA. Conversely, fine-tuning the MHA layers (MHA-FT) further perturbs the model’s language representations by entangling them with the time series signal, thereby reducing pseudo-alignment. Moreover, any random initialization within these components significantly degrades forecasting performance: LN-PF & MHA-RF yields the greatest performance drop, whereas LN-RT & MHA-PF incurs the smallest, indicating that most language knowledge is preserved in the MHA layer. Together, these findings demonstrate that the interaction between pretrained LN and MHA is the principal driver of pseudo-alignment, with LN-PT & MHA-PF causing the worst pseudo-alignment, as shown in Figure 2a.

7 Conclusion

In this work, we close the gap that limited insights and solution regarding pseudo-alignment problem by implementing comprehensive theoretical analysis and proposing an effective “plug-and-play” time enhancer module to address pseudo-alignment. We showed that Pseudo-Alignment problem is a synthetic effect of cone effect and low data manifold of time series data, building up the bridge of cone effect in LLM to time series data. With inspiration from the analysis, we proposed TimeSUP module to effectively mitigate this issue and consistently outperforms other LLM- and lightweight-based time series forecaster. Further visual evidence provided in this work indicates that the strong effectiveness of proposed solution to mitigate the pseudo-alignment problem.

8 Acknowledgment

This work was supported by Australian Research Council ARC Early Career Industry Fellowship (Grant No.IE240100275), Australian Research Council Discovery Projects (Grant No.DP240103070), Australian Research Council Linkage(Grant No.LP230200821), and Australia’s Economic Accelerator Ignite (Grant No. IG250200014).

References

- [1] Daniel Biś, Maksim Podkorytov, and Xiuwen Liu. 2021. Too much in common: Shifting of embeddings in transformer language models and its implications. In *Proceedings of the 2021 conference of the North American chapter of the Association for Computational Linguistics: Human Language Technologies*. 5117–5130.
- [2] Lawrence Cayton et al. 2005. Algorithms for manifold learning. *Univ. of California at San Diego Tech. Rep* 12, 1-17 (2005), 1.
- [3] Weitong Chen, Guodong Long, Lina Yao, and Quan Z Sheng. 2020. AMRNN: attended multi-task recurrent neural networks for dynamic illness severity prediction. *World Wide Web* 23, 5 (2020), 2753–2770.
- [4] Weitong Chen, Lin Yue, Bohan Li, Can Wang, and Quan Z Sheng. 2019. DAMTRNN: a delta attention-based multi-task RNN for intention recognition. In *International Conference on Advanced Data Mining and Applications*. Springer, 373–388.
- [5] Gamaleldin F Elsayed, Ian Goodfellow, and Jascha Sohl-Dickstein. 2018. Adversarial reprogramming of neural networks. *arXiv preprint arXiv:1806.11146* (2018).
- [6] Matthias Englert and Ranko Lazic. 2022. Adversarial reprogramming revisited. *Advances in Neural Information Processing Systems* 35 (2022), 28588–28600.
- [7] Jun Gao, Di He, Xu Tan, Tao Qin, Liwei Wang, and Tie-Yan Liu. 2019. Representation degeneration problem in training natural language generation models. *arXiv preprint arXiv:1907.12009* (2019).
- [8] Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew G Wilson. 2023. Large language models are zero-shot time series forecasters. *Advances in Neural Information Processing Systems* 36 (2023), 19622–19635.
- [9] Yuxiao Hu, Qian Li, Dongxiao Zhang, Jinyue Yan, and Yuntian Chen. 2025. Context-Alignment: Activating and Enhancing LLM Capabilities in Time Series. *arXiv preprint arXiv:2501.03747* (2025).
- [10] Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F Schmidt, Jonathan Weber, Geoffrey I Webb, Hassane Idoumghar, Pierre-Alain Muller, and François Petitjean. 2020. Inceptiontime: Finding alexnet for time series classification. *Data Mining and Knowledge Discovery* 34, 6 (2020), 1936–1962.
- [11] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. 2023. Time-llm: Time series forecasting by reprogramming large language models. *arXiv preprint arXiv:2310.01728* (2023).
- [12] Victor Weixin Liang, Yuhui Zhang, Yongchan Kwon, Serena Yeung, and James Y Zou. 2022. Mind the gap: Understanding the modality gap in multi-modal contrastive representation learning. *Advances in Neural Information Processing Systems* 35 (2022), 17612–17625.
- [13] Chenxi Liu, Qianxiong Xu, Hao Miao, Sun Yang, Lingzheng Zhang, Cheng Long, Ziyue Li, and Rui Zhao. 2025. Timecma: Towards llm-empowered multivariate time series forecasting via cross-modality alignment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 18780–18788.
- [14] Guangcan Liu, Zhouchen Lin, Shuicheng Yan, Ju Sun, Yong Yu, and Yi Ma. 2012. Robust recovery of subspace structures by low-rank representation. *IEEE transactions on pattern analysis and machine intelligence* 35, 1 (2012), 171–184.
- [15] Peiyuan Liu, Hang Guo, Tao Dai, Naiqi Li, Jigang Bao, Xudong Ren, Yong Jiang, and Shu-Tao Xia. 2025. Calf: Aligning llms for time series forecasting via cross-modal fine-tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 18915–18923.
- [16] Xu Liu, Junfeng Hu, Yuan Li, Shizhe Diao, Yuxuan Liang, Bryan Hooi, and Roger Zimmermann. 2024. Unitime: A language-empowered unified model for cross-domain time series forecasting. In *Proceedings of the ACM Web Conference 2024*. 4095–4106.
- [17] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2023. itransformer: Inverted transformers are effective for time series forecasting. *arXiv preprint arXiv:2310.06625* (2023).
- [18] Manfred Mudelsee. 2010. Climate time series analysis. *Atmospheric and* 397 (2010).
- [19] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2022. A time series is worth 64 words: Long-term forecasting with transformers. *arXiv preprint arXiv:2211.14730* (2022).
- [20] Zijie Pan, Yushan Jiang, Sahil Garg, Anderson Schneider, Yuriy Nemmyvka, and Dongjin Song. 2024. S²IP-LLM: Semantic Space Informed Prompt Learning with LLM for Time Series Forecasting. *arXiv:2403.05798* [cs.LG] <https://arxiv.org/abs/2403.05798>
- [21] Hussein Sharadga, Shima Hajimirza, and Robert S Balog. 2020. Time series forecasting of solar power generation for large-scale photovoltaic plants. *Renewable Energy* 150 (2020), 797–807.
- [22] Mustafa Shukor and Matthieu Cord. 2024. Implicit multimodal alignment: On the generalization of frozen llms to multimodal inputs. *Advances in Neural Information Processing Systems* 37 (2024), 130848–130886.
- [23] Mingtian Tan, Mike Merrill, Vinayak Gupta, Tim Althoff, and Tom Hartvigsen. 2024. Are language models actually useful for time series forecasting? *Advances in Neural Information Processing Systems* 37 (2024), 60162–60191.
- [24] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. 2025. Hydrarag: Structured cross-source enhanced large language model reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 14442–14470.
- [25] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. 2025. MemoTime: Memory-Augmented Temporal Knowledge Graph Enhanced Large Language Model Reasoning. *arXiv preprint arXiv:2510.13614* (2025).
- [26] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y Zhang, and Jun Zhou. 2024. Timemixer: Decomposable multiscale mixing for time series forecasting. *arXiv preprint arXiv:2405.14616* (2024).
- [27] Ruofeng Wen, Kari Torkkola, Balakrishnan Narayanaswamy, and Dhruv Madeka. 2017. A multi-horizon quantile recurrent forecaster. *arXiv preprint arXiv:1711.11053* (2017).
- [28] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2022. Timesnet: Temporal 2d-variation modeling for general time series analysis. *arXiv preprint arXiv:2210.02186* (2022).
- [29] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems* 34 (2021), 22419–22430.
- [30] Zhijian Xu, Ailing Zeng, and Qiang Xu. 2023. FITS: Modeling time series with 10k parameters. *arXiv preprint arXiv:2307.03756* (2023).
- [31] Chao-Han Huck Yang, Yun-Yun Tsai, and Pin-Yu Chen. 2021. Voice2series: Reprogramming acoustic models for time series classification. In *International conference on machine learning*. PMLR, 11808–11819.
- [32] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 11121–11128.
- [33] Jianhu Zheng and Mingfang Huang. 2020. Traffic flow forecast through time series analysis based on deep learning. *IEEE access* 8 (2020), 82562–82570.
- [34] Liangwei Nathan Zheng, Chang Dong, Wei Emma Zhang, Lin Yue, Miao Xu, Olaf Maennel, and Weitong Chen. 2025. Understanding why large language models can be ineffective in time series analysis: The impact of modality alignment. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 4026–4037.
- [35] Liangwei Nathan Zheng, Zhengyang Li, Chang George Dong, Wei Emma Zhang, Lin Yue, Miao Xu, Olaf Maennel, and Weitong Chen. 2024. Irregularity-Informed Time Series Analysis: Adaptive Modelling of Spatial and Temporal Dynamics. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 3405–3414.
- [36] Liangwei Nathan Zheng, Wei Emma Zhang, Mingyu Guo, Miao Xu, Olaf Maennel, and Weitong Chen. 2025. Rethinking Gating Mechanism in Sparse MoE: Handling Arbitrary Modality Inputs with Confidence-Guided Gate. *arXiv preprint arXiv:2505.19525* (2025).
- [37] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International conference on machine learning*. PMLR, 27268–27286.
- [38] Tian Zhou, Peisong Niu, Liang Sun, Rong Jin, et al. 2023. One fits all: Power general time series analysis by pretrained lm. *Advances in neural information processing systems* 36 (2023), 43322–43355.

A Additional Experiment Details

Table 3: Dataset statistics are from [11]. The dimension indicates the number of time series channel and the size is organized as (training, validation, testing).

Dataset	Dim.	Prediction Length	Dataset Size	Frequency	Domain
ETTh1	7	96, 192, 336, 720	(8545, 2881, 2881)	1 hour	Temperature
ETTh2	7	96, 192, 336, 720	(8545, 2881, 2881)	1 hour	Temperature
ETTm1	7	96, 192, 336, 720	(34465, 11521, 11521)	15 mins	Temperature
ETTm2	7	96, 192, 336, 720	(34465, 11521, 11521)	15 mins	Temperature
Illness	7	24, 36, 48, 60	(617, 74, 170)	1 week	Healthcare
Weather	21	96, 192, 336, 720	(36792, 5271, 10540)	10 mins	Weather
Traffic	862	96, 192, 336, 720	(12185, 1757, 3509)	1 hour	Transportation
ECL	862	96, 192, 336, 720	(18317, 2633, 5261)	1 hour	Electricity

The configurations of our model various to datasets while most configurations are consistent. By default, the Adam optimizer and Mean Square Error loss function are applied to trained the proposed method. The parameter described in Table 4 are the best parameter to reproduce the main results in Table 1.

Table 4: Hyperparameter Setting for results reproduction

Dataset	Input Length	Patch Size	Stride	Dropout	Learn. Rate	Comp. Token	Top-k Prototype	# LLM layer
ETTh1	336	16	8	0.3	1e-4	10	4	6
ETTh2	512	16	8	0.2	5e-4	32	4	6
ETTm1	512	16	16	0.2	1e-4	32	4	6
ETTm2	512	16	16	0.3	1e-4	32	6	6
Illness	104	24	2	0	1e-4	8	4	6
Weather	336	16	8	0.1	1e-4	16	4	6
Traffic	336	16	8	0	1e-3	10	4	6
ECL	336	16	8	0.1	1e-4	16	4	6

We found that FCSA, CALF S2IP and TimeLLM are the competitive model we have in the table. Therefore, to show the effectiveness of our method, we show results with variance in Table 5 under 3 different random seeds, which is the same source of results as Table 1, but with standard deviation to show that it is actually outperform others statistically.

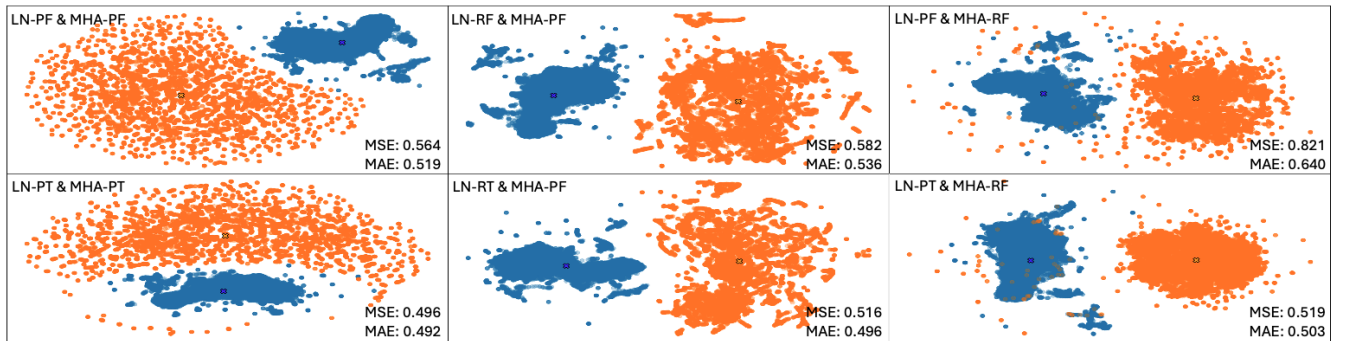


Figure 5: UMAP Representation for Different Setting on ETTh1-720 dataset. Blue is language and Orange is time series. Notation: LayerNorm(LN), MultiHead Attention(MHA), Pretrained–Frozen (PF), Pretrained–fineTuned (PT), Random–Frozen (RF), and Random–fineTuned (RT).

Dataset		TimeSUP		FSCA		CALF		S2IP		TimeLLM	
Pred Length		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.376 $\pm 6e^{-3}$	0.400 $\pm 4e^{-3}$	<u>0.381</u> $\pm 4e^{-3}$	<u>0.404</u> $\pm 7e^{-3}$	0.386 $\pm 4e^{-3}$	0.410 $\pm 8e^{-3}$	0.394 $\pm 11e^{-3}$	0.415 $\pm 10e^{-3}$	0.383 $\pm 10e^{-3}$	0.412 $\pm 8e^{-3}$
	192	0.408 $\pm 7e^{-3}$	0.427 $\pm 10e^{-3}$	<u>0.416</u> $\pm 12e^{-3}$	<u>0.419</u> $\pm 11e^{-3}$	0.431 $\pm 18e^{-3}$	0.430 $\pm 14e^{-3}$	0.443 $\pm 8e^{-3}$	0.445 $\pm 8e^{-3}$	0.424 $\pm 6e^{-3}$	0.434 $\pm 15e^{-3}$
	336	0.429 $\pm 12e^{-3}$	0.436 $\pm 10e^{-3}$	0.455 $\pm 7e^{-3}$	0.447 $\pm 11e^{-3}$	0.477 $\pm 6e^{-3}$	0.450 $\pm 6e^{-3}$	0.456 $\pm 7e^{-3}$	0.456 $\pm 13e^{-3}$	<u>0.437</u> $\pm 11e^{-3}$	0.453 $\pm 13e^{-3}$
	720	0.433 $\pm 2e^{-3}$	0.456 $\pm 3e^{-3}$	0.455 $\pm 4e^{-3}$	0.471 $\pm 4e^{-3}$	0.473 $\pm 3e^{-3}$	0.466 $\pm 3e^{-3}$	0.471 $\pm 1e^{-3}$	0.478 $\pm 4e^{-3}$	0.458 $\pm 3e^{-3}$	0.484 $\pm 3e^{-3}$
	Avg	0.412 $\pm 22e^{-3}$	0.430 $\pm 20e^{-3}$	<u>0.426</u> $\pm 30e^{-3}$	<u>0.435</u> $\pm 25e^{-3}$	0.441 $\pm 37e^{-3}$	0.439 $\pm 21e^{-3}$	0.441 $\pm 29e^{-3}$	0.448 $\pm 23e^{-3}$	<u>0.426</u> $\pm 27e^{-3}$	0.445 $\pm 26e^{-3}$
ETTh2	96	0.293 $\pm 8e^{-3}$	0.346 $\pm 5e^{-3}$	0.300 $\pm 10e^{-3}$	<u>0.351</u> $\pm 8e^{-3}$	0.297 $\pm 5e^{-3}$	<u>0.351</u> $\pm 8e^{-3}$	0.300 $\pm 7e^{-3}$	0.354 $\pm 6e^{-3}$	<u>0.295</u> $\pm 10e^{-3}$	0.353 $\pm 11e^{-3}$
	192	0.342 $\pm 7e^{-3}$	0.382 $\pm 10e^{-3}$	<u>0.348</u> $\pm 7e^{-3}$	<u>0.387</u> $\pm 9e^{-3}$	0.369 $\pm 8e^{-3}$	0.388 $\pm 9e^{-3}$	0.370 $\pm 12e^{-3}$	0.397 $\pm 10e^{-3}$	0.410 $\pm 8e^{-3}$	0.421 $\pm 11e^{-3}$
	336	<u>0.369</u> $\pm 8e^{-3}$	<u>0.411</u> $\pm 6e^{-3}$	0.365 $\pm 10e^{-3}$	0.405 $\pm 11e^{-3}$	0.422 $\pm 12e^{-3}$	0.426 $\pm 18e^{-3}$	0.401 $\pm 13e^{-3}$	0.426 $\pm 14e^{-3}$	0.423 $\pm 7e^{-3}$	0.441 $\pm 8e^{-3}$
	720	0.407 $\pm 3e^{-3}$	0.438 $\pm 5e^{-3}$	0.407 $\pm 4e^{-3}$	<u>0.444</u> $\pm 4e^{-3}$	<u>0.421</u> $\pm 4e^{-3}$	0.450 $\pm 6e^{-3}$	0.428 $\pm 7e^{-3}$	0.455 $\pm 7e^{-3}$	0.455 $\pm 3e^{-3}$	0.467 $\pm 4e^{-3}$
	Avg	0.353 $\pm 41e^{-3}$	0.394 $\pm 34e^{-3}$	<u>0.355</u> $\pm 38e^{-3}$	<u>0.397</u> $\pm 33e^{-3}$	0.376 $\pm 51e^{-3}$	0.404 $\pm 38e^{-3}$	0.374 $\pm 48e^{-3}$	0.408 $\pm 37e^{-3}$	0.396 $\pm 60e^{-3}$	0.420 $\pm 42e^{-3}$
ETTm1	96	0.296 $\pm 4e^{-3}$	0.350 $\pm 6e^{-3}$	<u>0.301</u> $\pm 8e^{-3}$	<u>0.351</u> $\pm 11e^{-3}$	0.324 $\pm 8e^{-3}$	0.353 $\pm 6e^{-3}$	0.300 $\pm 5e^{-3}$	0.353 $\pm 4e^{-3}$	0.304 $\pm 4e^{-3}$	0.361 $\pm 4e^{-3}$
	192	0.332 $\pm 7e^{-3}$	0.369 $\pm 10e^{-3}$	0.332 $\pm 9e^{-3}$	<u>0.371</u> $\pm 11e^{-3}$	0.369 $\pm 8e^{-3}$	0.376 $\pm 9e^{-3}$	<u>0.333</u> $\pm 7e^{-3}$	0.377 $\pm 10e^{-3}$	0.358 $\pm 5e^{-3}$	0.384 $\pm 5e^{-3}$
	336	0.360 $\pm 4e^{-3}$	0.385 $\pm 5e^{-3}$	0.365 $\pm 4e^{-3}$	<u>0.391</u> $\pm 4e^{-3}$	0.407 $\pm 4e^{-3}$	0.399 $\pm 8e^{-3}$	<u>0.362</u> $\pm 7e^{-3}$	0.396 $\pm 6e^{-3}$	0.364 $\pm 6e^{-3}$	0.397 $\pm 3e^{-3}$
	720	<u>0.411</u> $\pm 6e^{-3}$	0.418 $\pm 5e^{-3}$	0.421 $\pm 7e^{-3}$	<u>0.425</u> $\pm 4e^{-3}$	0.472 $\pm 6e^{-3}$	0.436 $\pm 11e^{-3}$	0.407 $\pm 8e^{-3}$	0.429 $\pm 7e^{-3}$	0.424 $\pm 5e^{-3}$	0.431 $\pm 4e^{-3}$
	Avg	<u>0.350</u> $\pm 42e^{-3}$	0.380 $\pm 25e^{-3}$	0.354 $\pm 44e^{-3}$	<u>0.385</u> $\pm 54e^{-3}$	0.393 $\pm 31e^{-3}$	0.391 $\pm 31e^{-3}$	0.349 $\pm 39e^{-3}$	0.389 $\pm 28e^{-3}$	0.363 $\pm 43e^{-3}$	0.389 $\pm 25e^{-3}$
ETTm2	96	<u>0.174</u> $\pm 3e^{-3}$	0.264 $\pm 6e^{-3}$	0.169 $\pm 7e^{-3}$	<u>0.260</u> $\pm 5e^{-3}$	0.178 $\pm 2e^{-3}$	0.255 $\pm 3e^{-3}$	0.181 $\pm 7e^{-3}$	0.269 $\pm 5e^{-3}$	0.178 $\pm 6e^{-3}$	0.269 $\pm 6e^{-3}$
	192	0.233 $\pm 10e^{-3}$	0.300 $\pm 6e^{-3}$	<u>0.237</u> $\pm 8e^{-3}$	0.306 $\pm 8e^{-3}$	0.243 $\pm 6e^{-3}$	0.298 $\pm 7e^{-3}$	0.257 $\pm 7e^{-3}$	0.323 $\pm 8e^{-3}$	0.241 $\pm 6e^{-3}$	0.313 $\pm 8e^{-3}$
	336	<u>0.280</u> $\pm 6e^{-3}$	0.338 $\pm 2e^{-3}$	0.276 $\pm 13e^{-3}$	0.328 $\pm 11e^{-3}$	0.303 $\pm 16e^{-3}$	0.338 $\pm 8e^{-3}$	0.293 $\pm 8e^{-3}$	0.346 $\pm 9e^{-3}$	0.287 $\pm 9e^{-3}$	<u>0.337</u> $\pm 8e^{-3}$
	720	0.358 $\pm 8e^{-3}$	0.381 $\pm 4e^{-3}$	0.366 $\pm 5e^{-3}$	<u>0.390</u> $\pm 4e^{-3}$	0.403 $\pm 4e^{-3}$	0.396 $\pm 5e^{-3}$	0.367 $\pm 3e^{-3}$	0.396 $\pm 5e^{-3}$	<u>0.364</u> $\pm 3e^{-3}$	0.401 $\pm 3e^{-3}$
	Avg	0.261 $\pm 67e^{-3}$	0.323 $\pm 44e^{-3}$	<u>0.262</u> $\pm 71e^{-3}$	0.321 $\pm 47e^{-3}$	0.282 $\pm 83e^{-3}$	<u>0.322</u> $\pm 52e^{-3}$	0.279 $\pm 67e^{-3}$	0.331 $\pm 46e^{-3}$	0.267 $\pm 68e^{-3}$	0.330 $\pm 48e^{-3}$
Illness	24	<u>1.942</u> $\pm 46e^{-3}$	0.835 $\pm 30e^{-3}$	2.105 $\pm 45e^{-3}$	0.908 $\pm 70e^{-3}$	2.271 $\pm 81e^{-3}$	1.008 $\pm 73e^{-3}$	2.622 $\pm 64e^{-3}$	1.113 $\pm 87e^{-3}$	1.921 $\pm 48e^{-3}$	<u>0.902</u> $\pm 69e^{-3}$
	36	1.952 $\pm 48e^{-3}$	0.948 $\pm 66e^{-3}$	2.066 $\pm 68e^{-3}$	0.962 $\pm 55e^{-3}$	2.365 $\pm 58e^{-3}$	1.029 $\pm 34e^{-3}$	2.515 $\pm 62e^{-3}$	1.098 $\pm 46e^{-3}$	<u>1.992</u> $\pm 57e^{-3}$	0.918 $\pm 48e^{-3}$
	48	1.738 $\pm 83e^{-3}$	0.886 $\pm 71e^{-3}$	<u>1.902</u> $\pm 90e^{-3}$	0.906 $\pm 97e^{-3}$	2.395 $\pm 24e^{-3}$	1.046 $\pm 39e^{-3}$	2.530 $\pm 22e^{-3}$	1.066 $\pm 37e^{-3}$	2.187 $\pm 31e^{-3}$	1.031 $\pm 12e^{-3}$
	60	1.907 $\pm 34e^{-3}$	0.942 $\pm 38e^{-3}$	2.341 $\pm 46e^{-3}$	1.016 $\pm 27e^{-3}$	2.415 $\pm 36e^{-3}$	1.056 $\pm 20e^{-3}$	2.329 $\pm 21e^{-3}$	1.065 $\pm 14e^{-3}$	2.125 $\pm 27e^{-3}$	<u>0.992</u> $\pm 21e^{-3}$
	Avg	1.885 $\pm 86e^{-3}$	0.902 $\pm 46e^{-3}$	2.103 $\pm 157e^{-3}$	0.948 $\pm 45e^{-3}$	2.361 $\pm 55e^{-3}$	1.035 $\pm 18e^{-3}$	2.387 $\pm 106e^{-3}$	1.052 $\pm 21e^{-3}$	<u>2.056</u> $\pm 102e^{-3}$	0.961 $\pm 53e^{-3}$
Weather	96	0.151 $\pm 7e^{-3}$	<u>0.200</u> $\pm 3e^{-3}$	<u>0.152</u> $\pm 2e^{-3}$	0.201 $\pm 4e^{-3}$	0.163 $\pm 6e^{-3}$	0.203 $\pm 9e^{-3}$	0.157 $\pm 8e^{-3}$	0.209 $\pm 8e^{-3}$	0.154 $\pm 7e^{-3}$	0.199 $\pm 6e^{-3}$
	192	0.197 $\pm 7e^{-3}$	0.243 $\pm 5e^{-3}$	0.197 $\pm 5e^{-3}$	<u>0.244</u> $\pm 4e^{-3}$	0.210 $\pm 4e^{-3}$	0.248 $\pm 4e^{-3}$	<u>0.200</u> $\pm 11e^{-3}$	0.251 $\pm 9e^{-3}$	0.214 $\pm 7e^{-3}$	0.254 $\pm 8e^{-3}$
	336	0.248 $\pm 10e^{-3}$	0.284 $\pm 13e^{-3}$	<u>0.253</u> $\pm 7e^{-3}$	<u>0.289</u> $\pm 5e^{-3}$	0.266 $\pm 6e^{-3}$	0.290 $\pm 5e^{-3}$	<u>0.253</u> $\pm 7e^{-3}$	0.288 $\pm 8e^{-3}$	0.273 $\pm 6e^{-3}$	0.291 $\pm 11e^{-3}$
	720	0.330 $\pm 3e^{-3}$	<u>0.340</u> $\pm 2e^{-3}$	<u>0.332</u> $\pm 4e^{-3}$	<u>0.341</u> $\pm 7e^{-3}$	0.347 $\pm 6e^{-3}$	0.344 $\pm 6e^{-3}$	<u>0.338</u> $\pm 2e^{-3}$	0.333 $\pm 4e^{-3}$	0.364 $\pm 7e^{-3}$	0.350 $\pm 4e^{-3}$
	Avg	0.231 $\pm 66e^{-3}$	0.266 $\pm 46e^{-3}$	<u>0.233</u> $\pm 68e^{-3}$	<u>0.269</u> $\pm 52e^{-3}$	0.246 $\pm 68e^{-3}$	0.271 $\pm 52e^{-3}$	0.237 $\pm 68e^{-3}$	0.270 $\pm 29e^{-3}$	0.251 $\pm 78e^{-3}$	0.274 $\pm 54e^{-3}$
Traffic	96	0.369 $\pm 10e^{-3}$	0.266 $\pm 11e^{-3}$	<u>0.372</u> $\pm 6e^{-3}$	0.255 $\pm 9e^{-3}$	0.433 $\pm 5e^{-3}$	0.292 $\pm 7e^{-3}$	0.379 $\pm 2e^{-3}$	0.274 $\pm 5e^{-3}$	0.388 $\pm 4e^{-3}$	<u>0.261</u> $\pm 7e^{-3}$
	192	0.391 $\pm 4e^{-3}$	<u>0.277</u> $\pm 3e^{-3}$	0.399 $\pm 6e^{-3}$	0.284 $\pm 3e^{-3}$	0.451 $\pm 8e^{-3}$	0.297 $\pm 10e^{-3}$	<u>0.397</u> $\pm 9e^{-3}$	0.282 $\pm 11e^{-3}$	0.409 $\pm 7e^{-3}$	0.272 $\pm 7e^{-3}$
	336	0.408 $\pm 4e^{-3}$	<u>0.287</u> $\pm 3e^{-3}$	0.413 $\pm 5e^{-3}$	0.273 $\pm 7e^{-3}$	0.467 $\pm 10e^{-3}$	0.302 $\pm 11e^{-3}$	<u>0.410</u> $\pm 7e^{-3}$	0.289 $\pm 4e^{-3}$	0.438 $\pm 5e^{-3}$	0.299 $\pm 3e^{-3}$
	720	0.435 $\pm 3e^{-3}$	<u>0.298</u> $\pm 3e^{-3}$	<u>0.440</u> $\pm 4e^{-3}$	0.291 $\pm 2e^{-3}$	0.485 $\pm 7e^{-3}$	0.323 $\pm 5e^{-3}$	0.445 $\pm 4e^{-3}$	0.301 $\pm 8e^{-3}$	0.462 $\pm 6e^{-3}$	0.301 $\pm 4e^{-3}$
	Avg	0.400 $\pm 17e^{-3}$	<u>0.282</u> $\pm 12e^{-3}$	<u>0.406</u> $\pm 25e^{-3}$	0.276 $\pm 16e^{-3}$	0.459 $\pm 19e^{-3}$	0.303 $\pm 12e^{-3}$	0.407 $\pm 24e^{-3}$	0.287 $\pm 10e^{-3}$	0.424 $\pm 28e^{-3}$	0.283 $\pm 17e^{-3}$
ECL	96	0.128 $\pm 8e^{-3}$	0.221 $\pm 5e^{-3}$	0.133 $\pm 4e^{-3}$	0.228 $\pm 6e^{-3}$	0.167 $\pm 7e^{-3}$	0.254 $\pm 3e^{-3}$	<u>0.131</u> $\pm 4e^{-3}$	<u>0.226</u> $\pm 10e^{-3}$	0.172 $\pm 8e^{-3}$	0.265 $\pm 7e^{-3}$
	192	0.151 $\pm 9e^{-3}$	0.247 $\pm 5e^{-3}$	<u>0.157</u> $\pm 4e^{-3}$	<u>0.250</u> $\pm 3e^{-3}$	0.178 $\pm 5e^{-3}$	0.264 $\pm 4e^{-3}$	0.158 $\pm 3e^{-3}$	0.252 $\pm 3e^{-3}$	0.182 $\pm 6e^{-3}$	0.279 $\pm 5e^{-3}$
	336	0.170 $\pm 7e^{-3}$	0.269 $\pm 4e^{-3}$	<u>0.176</u> $\pm 10e^{-3}$	0.281 $\pm 13e^{-3}$	0.196 $\pm 5e^{-3}$	0.282 $\pm 7e^{-3}$	0.183 $\pm 6e^{-3}$	<u>0.280</u> $\pm 3e^{-3}$	0.195 $\pm 2e^{-3}$	0.288 $\pm 4e^{-3}$
	720	0.207 $\pm 4e^{-3}$	0.299 $\pm 3e^{-3}$	0.215 $\pm 5e^{-3}$	<u>0.302</u> $\pm 9e^{-3}$	0.232 $\pm 4e^{-3}$	0.312 $\pm 4e^{-3}$	<u>0.210</u> $\pm 4e^{-3}$	0.306 $\pm 3e^{-3}$	0.233 $\pm 7e^{-3}$	0.320 $\pm 5e^{-3}$
	Avg	0.164 $\pm 29e^{-3}$	0.256 $\pm 29e^{-3}$	<u>0.170</u> $\pm 30e^{-3}$	<u>0.265</u> $\pm 28e^{-3}$	0.193 $\pm 25e^{-3}$	0.278 $\pm 22e^{-3}$	<u>0.170</u> $\pm 29e^{-3}$	0.266 $\pm 30e^{-3}$	0.195 $\pm 23e^{-3}$	0.288 $\pm 20e^{-3}$
Wins		51		14		1		3		3	

Table 5: Full Results Long-term Forecasting with LLM4TS with std: Bold Text indicate the best performer and Underline Text indicate the second best performer. We count the number of wins by the number of best performer.

B Detailed Derivation

We provide detailed derivation of $E[\|x_{ts}\|^2]$ and $Var(\|x_{ts}\|)$

B.1 Eq.5

$$E[\|x_{ts}\|^2] = E[\|\mu_{ts} + \epsilon_{ts}\|^2] \quad (10)$$

$$= E[\|\mu_{ts}\|^2 + 2\langle \mu_{ts}, \epsilon_{ts} \rangle + \|\epsilon_{ts}\|^2] \quad (11)$$

$$= \|\mu_{ts}\|^2 + E[\|\epsilon_{ts}\|^2] = \|\mu_{ts}\|^2 + E\left[\sum_{i=1}^m \epsilon_{ts,i}^2\right] \quad (12)$$

$$= \|\mu_{ts}\|^2 + mE[\epsilon_{ts,i}^2] = \|\mu_{ts}\|^2 + m\frac{1}{m} \sum_{i=1}^m \epsilon_{ts,i}^2 \quad (13)$$

$$= \|\mu_{ts}\|^2 + m\sigma_{ts}^2 \quad (14)$$

Law of large number is applied at Eq. 13 as $m \rightarrow \infty$, therefore, $\frac{1}{m} \sum_{i=1}^m \epsilon_{ts,i}^2 \rightarrow \sigma_{ts}^2$. Thus, $\|x_{ts}\|^2 = \|\mu_{ts}\|^2 + m\sigma_{ts}^2$.

B.2 Eq.6

$$Var(\|x_{ts}\|^2) = Var(2\mu_{ts}\epsilon_{ts}) + Var(\|\epsilon_{ts}\|^2) \quad (15)$$

$$= 4\sigma_{ts}^2 \|\mu_{ts}\|^2 + Var\left(\sum_{i=1}^m v_i^2\right) \quad (16)$$

$$= 4\sigma_{ts}^2 \|\mu_{ts}\|^2 + m(3\sigma_{ts}^4 - \sigma_{ts}^4) \quad (17)$$

$$= 4\sigma_{ts}^2 \|\mu_{ts}\|^2 + 2m\sigma_{ts}^4 \quad (18)$$

In Eq.16, we applied the variance definition and the fourth moment of v_i from moment generating function.